Load data and Libraries :

```
import pandas as pd
import numpy as np

data = pd.read_csv("dirty_cafe_sales.csv")
print(data)
```

```
     Transaction ID      Item Quantity Price Per Unit Total Spent  \
0       TXN_1961373    Coffee        2            2.0         4.0
1       TXN_4977031      Cake        4            3.0        12.0
2       TXN_4271903    Cookie        4            1.0       ERROR
3       TXN_7034554     Salad        2            5.0        10.0
4       TXN_3160411    Coffee        2            2.0         4.0
...             ...       ...      ...            ...         ...
9995    TXN_7672686    Coffee        2            2.0         4.0
9996    TXN_9659401       NaN        3            NaN         3.0
9997    TXN_5255387    Coffee        4            2.0         8.0
9998    TXN_7695629    Cookie        3            NaN         3.0
9999    TXN_6170729  Sandwich        3            4.0        12.0

      Payment Method  Location Transaction Date
0        Credit Card  Takeaway       2023-09-08
1               Cash  In-store       2023-05-16
2        Credit Card  In-store       2023-07-19
3            UNKNOWN   UNKNOWN       2023-04-27
4     Digital Wallet  In-store       2023-06-11
...              ...       ...              ...
9995             NaN   UNKNOWN       2023-08-30
9996  Digital Wallet       NaN       2023-06-02
9997  Digital Wallet       NaN       2023-03-02
9998  Digital Wallet       NaN       2023-12-02
9999            Cash  In-store       2023-11-07

[10000 rows x 8 columns]
```

```
data.dtypes
```

```
Transaction ID      str
Item                str
Quantity            str
Price Per Unit      str
Total Spent         str
Payment Method      str
Location            str
Transaction Date    str
dtype: object
```

```
data["Quantity"] = pd.to_numeric(
    data["Quantity"],
    errors="coerce"
).astype("Int64")

data["Transaction Date"] = pd.to_datetime(
    data["Transaction Date"],
    errors="coerce"
)

data["Price Per Unit"] = pd.to_numeric(
    data["Price Per Unit"],
    errors="coerce"
).astype(float)

data["Total Spent"] = pd.to_numeric(
    data["Total Spent"],
    errors="coerce"
).astype(float)

print(data["Quantity"].dtype)
print(data["Price Per Unit"].dtype)
print(data["Total Spent"].dtype)
print(data["Transaction Date"].dtype)
```

```
Int64
float64
float64
datetime64[us]
```

```
data.isnull().sum()
```

```
Transaction ID         0
Item                 333
Quantity             479
Price Per Unit       533
Total Spent          502
Payment Method      2579
Location            3265
Transaction Date     460
dtype: int64
```

```
missing_summary = pd.DataFrame({
    "Missing Count": data.isnull().sum(),
    "Missing Percentage": (
        data.isnull().sum() / len(data) * 100
    ).round(2)
})
```

```
print(missing_summary)
```

```
                  Missing Count  Missing Percentage
Transaction ID                0                0.00
Item                        333                3.33
Quantity                    479                4.79
Price Per Unit              533                5.33
Total Spent                 502                5.02
Payment Method             2579               25.79
Location                   3265               32.65
Transaction Date            460                4.60
```

```
import pandas as pd

#Convert numeric columns

numeric_columns = [
    "Quantity",
    "Price Per Unit",
    "Total Spent"
]

for column in numeric_columns :
    data[column] = pd.to_numeric ( data[column],errors ="coerce")

data["Transaction Date"] = pd.to_datetime(
    data["Transaction Date"],errors = "coerce")

# Fill categorical Columns Using "known Words"
data["Item"] = data["Item"].fillna("Unknown")
data["Payment Method"] = data["Payment Method"].fillna("Unknown")
data["Location"] = data["Location"].fillna("Not defined")


#Fill Numerical Columns Using Mean,Median and Mode Values

data["Quantity"] = data["Quantity"].fillna(
    data["Quantity"].median()
)

data["Price Per Unit"] = data["Price Per Unit"].fillna(
    data["Price Per Unit"].median()
)

data.duplicated().sum()
```

```
0
```

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    10000 non-null  str
 1   Item              9667 non-null   str
 2   Quantity          9521 non-null   Int64
 3   Price Per Unit    9467 non-null   float64
 4   Total Spent       9498 non-null   float64
 5   Payment Method    7421 non-null   str
 6   Location          6735 non-null   str
 7   Transaction Date  9540 non-null   datetime64[us]
dtypes: Int64(1), datetime64[us](1), float64(2), str(4)
memory usage: 918.6 KB

data.describe()

       Quantity  Price Per Unit  Total Spent             Transaction
Date
count    9521.0     9467.000000  9498.000000
9540
mean   3.028463        2.949984     8.924352  2023-07-01
23:00:31.698113
min         1.0        1.000000     1.000000         2023-01-01
00:00:00
25%         2.0        2.000000     4.000000         2023-04-01
00:00:00
50%         3.0        3.000000     8.000000         2023-07-02
00:00:00
75%         4.0        4.000000    12.000000         2023-10-02
00:00:00
max         5.0        5.000000    25.000000         2023-12-31
00:00:00
std    1.419007        1.278450     6.009919
NaN

# Check invalid quantities
invalid_quantity = data[
    data["Quantity"] <= 0
]

# Check invalid prices
invalid_price = data[
    data["Price Per Unit"] <= 0
]

# Check invalid total amounts
invalid_total_amount = data[
    data["Total Spent"] < 0
```

```
]

print("Invalid Quantity records:")
print(invalid_quantity)

print("Invalid Price records:")
print(invalid_price)

print("Invalid Total Amount records:")
print(invalid_total_amount)
```

```
Invalid Quantity records:
Empty DataFrame
Columns: [Transaction ID, Item, Quantity, Price Per Unit, Total Spent,
Payment Method, Location, Transaction Date]
Index: []
Invalid Price records:
Empty DataFrame
Columns: [Transaction ID, Item, Quantity, Price Per Unit, Total Spent,
Payment Method, Location, Transaction Date]
Index: []
Invalid Total Amount records:
Empty DataFrame
Columns: [Transaction ID, Item, Quantity, Price Per Unit, Total Spent,
Payment Method, Location, Transaction Date]
Index: []
```

```
invalid_dates = data[data["Transaction Date"].isna()]

print(invalid_dates)
```

```
     Transaction ID      Item  Quantity  Price Per Unit  Total
Spent  \
11      TXN_3051279  Sandwich         2             4.0          8.0

29      TXN_7640952      Cake         4             3.0         12.0

33      TXN_7710508   UNKNOWN         5             1.0          5.0

77      TXN_2091733     Salad         1             5.0          5.0

103     TXN_7028009      Cake         4             3.0         12.0

...             ...       ...       ...             ...          ...

9933    TXN_9460419      Cake         1             3.0          3.0

9937    TXN_8253472      Cake         1             3.0          3.0

9949    TXN_3130865     Juice         3             3.0          9.0
```

```
9983    TXN_9226047  Smoothie         3             4.0          12.0

9988    TXN_9594133      Cake         5             3.0           NaN


      Payment Method  Location Transaction Date
11       Credit Card  Takeaway              NaT
29    Digital Wallet  Takeaway              NaT
33              Cash       NaN              NaT
77               NaN  In-store              NaT
103              NaN  Takeaway              NaT
...              ...       ...              ...
9933             NaN  Takeaway              NaT
9937             NaN       NaN              NaT
9949             NaN  In-store              NaT
9983            Cash       NaN              NaT
9988           ERROR       NaN              NaT

[460 rows x 8 columns]
```

```
invalid_sales = data[data["Total Spent"].isna()]

print(invalid_sales)
```

```
     Transaction ID      Item  Quantity  Price Per Unit  Total
Spent  \
2       TXN_4271903    Cookie         4             1.0           NaN

25      TXN_7958992  Smoothie         3             4.0           NaN

31      TXN_8927252   UNKNOWN         2             1.0           NaN

42      TXN_6650263       Tea         2             1.5           NaN

65      TXN_4987129  Sandwich         3             NaN           NaN

...             ...       ...       ...             ...           ...

9893    TXN_3809533     Juice         2             NaN           NaN

9954    TXN_1191659    Coffee         4             2.0           NaN

9977    TXN_5548914     Juice         2             3.0           NaN

9988    TXN_9594133      Cake         5             3.0           NaN

9993    TXN_4766549  Smoothie         2             4.0           NaN


      Payment Method  Location Transaction Date
2        Credit Card  In-store       2023-07-19
```

```
25           UNKNOWN   UNKNOWN       2023-12-13
31       Credit Card     ERROR       2023-11-06
42               NaN  Takeaway       2023-01-10
65               NaN  In-store       2023-10-20
...              ...       ...              ...
9893  Digital Wallet  Takeaway       2023-02-02
9954     Credit Card  In-store       2023-11-21
9977  Digital Wallet  In-store       2023-11-04
9988           ERROR       NaN              NaT
9993            Cash       NaN       2023-10-20

[502 rows x 8 columns]
```

```
print("Invalid quantities:", (data["Quantity"] <= 0).sum())
print("Invalid prices:", (data["Price Per Unit"] <= 0).sum())
print("Invalid total amounts:", (data["Total Spent"] < 0).sum())
```

```
Invalid quantities: 0
Invalid prices: 0
Invalid total amounts: 0
```

```
print(data.duplicated().any())
```

```
False
```

```
unique_customers = data["Transaction ID"].nunique()

print(unique_customers)
```

```
10000
```

```
Calculated_total = (
    data["Quantity"] * data["Price Per Unit"]
)

data["Total Spent"] = data["Total Spent"].fillna(Calculated_total)

data["Location"] = data["Location"].replace("UNKNOWN","Unknown")
data["Payment Method"] = data["Payment Method"].replace("UNKNOWN" ,
"Not defined")

data
```

```
     Transaction ID      Item  Quantity  Price Per Unit  Total
Spent  \
0       TXN_1961373    Coffee         2             2.0          4.0

1       TXN_4977031      Cake         4             3.0         12.0

2       TXN_4271903    Cookie         4             1.0          4.0

3       TXN_7034554     Salad         2             5.0         10.0
```

```
4       TXN_3160411    Coffee         2             2.0           4.0
...             ...       ...       ...             ...           ...
9995    TXN_7672686    Coffee         2             2.0           4.0
9996    TXN_9659401   Unknown         3             3.0           3.0
9997    TXN_5255387    Coffee         4             2.0           8.0
9998    TXN_7695629    Cookie         3             3.0           3.0
9999    TXN_6170729  Sandwich         3             4.0          12.0
```

```
      Payment Method     Location Transaction Date
0        Credit Card     Takeaway       2023-09-08
1               Cash     In-store       2023-05-16
2        Credit Card     In-store       2023-07-19
3        Not defined      Unknown       2023-04-27
4     Digital Wallet     In-store       2023-06-11
...              ...          ...              ...
9995         Unknown      Unknown       2023-08-30
9996  Digital Wallet  Not defined       2023-06-02
9997  Digital Wallet  Not defined       2023-03-02
9998  Digital Wallet  Not defined       2023-12-02
9999            Cash     In-store       2023-11-07

[10000 rows x 8 columns]
```

```
# Checking the data which is correctly replace with descriptive
statistics methods like median
value = data.loc[9893,"Price Per Unit"]
print(value)
```

```
3.0
```

```
subset = data.loc[[45,65,9893],["Price Per Unit","Total
Spent","Quantity"]]
print(subset)
```

```
      Price Per Unit  Total Spent  Quantity
45               5.0         15.0         3
65               3.0          9.0         3
9893             3.0          6.0         2
```

Finding Outliers Using IQR and Z-Score

```
Q1 = data["Total Spent"].quantile(0.25)
Q3 = data["Total Spent"].quantile(0.75)

IQR = Q3-Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1 :",Q1)
print("Q3 :",Q3)
print("IQR :",IQR)
print("Lower Bound :",lower_bound)
print("Upper Bound :",upper_bound)
```

```
Q1 : 4.0
Q3 : 12.0
IQR : 8.0
Lower Bound : -8.0
Upper Bound : 24.0
```

```
Q1 = data["Price Per Unit"].quantile(0.25)
Q3 = data["Price Per Unit"].quantile(0.75)

IQR = Q3-Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1 :",Q1)
print("Q3 :",Q3)
print("IQR :",IQR)
print("Lower Bound :",lower_bound)
print("Upper Bound :",upper_bound)
```

```
Q1 : 2.0
Q3 : 4.0
IQR : 2.0
Lower Bound : -1.0
Upper Bound : 7.0
```

```
# Identify Outliers using the 1.5 X IQR Rule :
outliers_iqr = data[
    (data["Total Spent"] < lower_bound) |
    (data["Total Spent"] > upper_bound)
]

print("Number of IQR outliers:", len(outliers_iqr))
print(outliers_iqr[["Total Spent"]].head())
```

```
Number of IQR outliers: 5322
   Total Spent
1         12.0
3         10.0
5         20.0
6          9.0
7         16.0
```

```
#Calculate z-score for a column
#Method 1: Manual formula

mean_value = data["Price Per Unit"].mean()
std_value = data["Price Per Unit"].std()

data["Price_Per_Unit_zscore"] = (
    data["Price Per Unit"] - mean_value
) / std_value

print(data[["Price Per Unit", "Price_Per_Unit_zscore"]].head())
```

```
   Price Per Unit  Price_Per_Unit_zscore
0             2.0              -0.765820
1             3.0               0.038064
2             1.0              -1.569704
3             5.0               1.645832
4             2.0              -0.765820
```

```
#Calculate z-score for a column
#Method 1: Manual formula

mean_value = data["Total Spent"].mean()
std_value = data["Total Spent"].std()

data["Total_Spent_zscore"] = (
    data["Total Spent"] - mean_value
) / std_value

print(data[["Total Spent", "Total_Spent_zscore"]].head())
```

```
   Total Spent  Total_Spent_zscore
0          4.0           -0.822186
1         12.0            0.511787
2          4.0           -0.822186
3         10.0            0.178294
4          4.0           -0.822186
```